

Short-circuiting and switch

CSC103 Programming Fundamentals / Lecture 09

Dr. Muhammad Farhan

Associate Professor of Computer Science
Department of Computer Science
COMSATS University Islamabad
Sahiwal Campus

Fall 2026

The problem for this lecture

Write a menu mapping 1 to Add, 2 to Remove and every other integer to Invalid. Test 1, 2 and 9.

Read one integer menu choice.

Learning objectives

- Use short-circuit guards safely
- Choose a conditional expression
- Implement a classic switch statement

CLO-2

Short-circuit evaluation

Conditional operator

Classic switch selection

Choice of selection structure

Short-circuit evaluation

- `&&` skips its right operand when the left operand is false.
- `||` skips its right operand when the left operand is true.
- A guard must precede the operation it protects.

Example

```
boolean safe = divisor != 0
    && numerator / divisor > 2;
```

Conditional operator

- condition ? valueIfTrue : valueIfFalse produces a value.
- Use it for short alternatives with compatible types.
- Use if-else for longer branch behaviour.

Example

```
int larger = a >= b ? a : b;
```

Classic switch selection

- switch selects a branch from a selector value.
- case labels are constant alternatives. default handles unmatched values.
- Classic colon cases continue into later cases unless control exits.

Example

```
switch (choice) {  
    case 1: label = "Open"; break;  
    case 2: label = "Save"; break;  
    default: label = "Unknown";  
}
```

Choice of selection structure

- if-else handles ranges and compound conditions.
- switch suits discrete alternatives such as menu choices.
- Document any intentional fall-through.

Example

Range: `temperature < 0`

Discrete choice: `menu option 1, 2 or 3`

A guard protects only later evaluation

- Short-circuiting follows left-to-right evaluation.
- A safe condition first establishes the requirement for the next operation.
- With integer division, the divisor check must precede the division.

Example

```
d != 0 && 10 / d > 1
```

For `d=0`, the right side never runs.

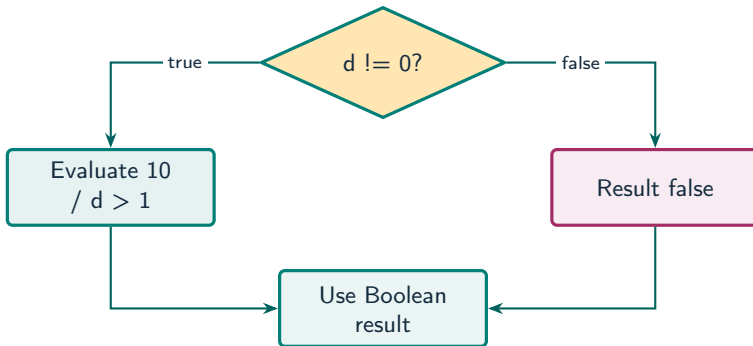
Switch fall-through is an execution rule

- A matching classic case determines where execution enters the switch.
- It does not automatically determine where execution leaves.
- `break` transfers control to the statement after the switch.

Example

```
case 1: print Add; break;  
case 2: print Remove; break;  
default: print Invalid;
```

A short-circuit guard controls evaluation



What to notice

When `d` is zero, Java skips the division and the entire AND expression is false.

Key distinctions

Construct	Best suited to	Key behaviour
if-else	Ranges and compound rules	First matching branch
?:	Short value alternatives	Produces a selected value
Classic switch	Discrete menu alternatives	Falls through without an exit

These distinctions determine which operation or control structure fits the problem.

First example: execution

```
int choice = 2;
String label;
switch (choice) {
    case 1: label = "Open"; break;
    case 2: label = "Save"; break;
    default: label = "Unknown";
}
System.out.println(label);
```

First example: result

Save
Execution enters case 2.
break exits the switch.

Classic switch selection
Choice of selection structure

Guided practice

Write a menu mapping 1 to Add, 2 to Remove and every other integer to Invalid. Test 1, 2 and 9.

Attempt the solution before continuing.
Use the definitions and examples from this lecture.

Solution strategy

- Read one integer menu choice.
- Use distinct case labels for 1 and 2, with a break in each.
- Use default to report every unmatched integer.

Sample console input
2

Complete solution (1/2)

```
public class Solution09 {  
    public static void main(String[] args) throws Exception {  
        java.util.Scanner in = new java.util.Scanner(System.in);  
        int choice = in.nextInt();  
        switch (choice) {  
            case 1:  
                System.out.println("Add");  
                break;  
        }  
    }  
}
```


Complete solution (2/2)

```
    case 2:  
        System.out.println("Remove");  
        break;  
    default:  
        System.out.println("Invalid");  
    }  
}  
}
```

Execution trace

Choice	Matching case	Output	Exit
1	case 1	Add	break
2	case 2	Remove	break
9	default	Invalid	End of switch

Follow the changing state in execution order.

Solution result

Remove

Why this result follows
Use default to report every
unmatched integer.

A common incorrect approach

```
boolean ok = 10 / d > 1 && d != 0;
```

Example

Put `d != 0` first. The current order divides before checking safety and **throws** when `d` is zero.

Boundary and failure checks

Check the stated input assumptions.

Create a four-option calculator using switch. Handle an invalid option and guard integer division by zero.

Use two case labels, each ending with break, plus a default branch.
Outputs: Add, Remove, Invalid.

Check your understanding

What does `false && riskyCall()` do? Why can a missing break change output?

Write a prediction and explain the rule that supports it.

Answer and explanation

It skips riskyCall. A missing break in a classic switch can let execution continue into later case bodies.

Put $d \neq 0$ first. The current order divides before checking safety and throws when d is zero.

Compare cases and predict outcomes

d	Right operand evaluated?	Result
0	No	false
2	Yes: $5 > 1$	true
10	Yes: $1 > 1$	false

Discuss

Explain each expected result before running the example. Which case would reveal a wrong assumption?

Apply the idea

Independent practice

Explain why reversing the operands in $d \neq 0 \ \&\& \ 10 / d > 1$ is unsafe. Write a guarded test and predict it for $d=0$, 2 and 10.

1. Work through the input by hand.
2. Write or correct the program.
3. Justify the expected result.

Solution and reasoning

```
boolean accepted = d != 0 && 10 / d > 1;
```

- Java evaluates the left operand first.
- Putting the division first can throw `ArithmeticException` before the zero check.
- The three results are false, true and false.

Letter grades as a switch problem

- A fixed mapping from one letter to one value fits discrete selection.
- Normalise case once, then reject letters outside the specified grade set.
- In classic switch syntax, break prevents unintended fall-through.

Source: [supplied ISB campus slides](#). See [speaker notes](#) and [ISB_Source_Map.md](#).

Worked problem: Letter grades as a switch problem

Task

Map A, B, C, D and F to 4, 3, 2, 1 and 0. Demonstrate lowercase b after case normalisation.

Input: Values are fixed in the program for a reproducible demonstration.

Before running

Predict the output and identify the variables that change.

Complete ISB example (1/1)

```
public class IsbLecture09 {  
    public static void main(String[] args) throws Exception {  
        char grade = Character.toUpperCase('b');  
        int points;  
        switch (grade) {  
            case 'A': points = 4; break;  
            case 'B': points = 3; break;  
            case 'C': points = 2; break;  
            case 'D': points = 1; break;  
            case 'F': points = 0; break;  
            default: points = -1;  
        }  
        System.out.println(points);  
    }  
}
```

Worked trace and expected result

Input	Normalised	Result
b	B	3
F	F	0
E	E	-1: invalid grade

Expected console output

3

Extend the ISB example

Practice

If input is a String rather than a char, what must be checked before `charAt(0)`?

Explain your reasoning

Require exactly one character; empty input and multi-character grades should be rejected.

Lecture summary

- short-circuit guards safely
 - a conditional expression
 - a classic switch statement
- Read one integer menu choice.
 - Use distinct case labels for 1 and 2, with a break in each.
 - Use default to report every unmatched integer.

After-class practice

Create a four-option calculator using switch. Handle an invalid option and guard integer division by zero.

Syllabus reading
Deitel Ch 7 (as listed)
Next lecture: While loops